

Multi-Agent Coding Workflow

Quy trình phát triển phần mềm đa tác tử, phiên bản cải tiến

1. Nguyên tắc cốt lõi

- **Người làm không tự chấm bài.** Agent thực hiện không được tự đánh giá kết quả của mình.
- **Không tin báo cáo "completed".** Chỉ tin bằng chứng kiểm chứng độc lập (raw output, diff, log).
- **Rẻ và xác định chạy trước, đắt chạy sau.** Build, test chạy trước review bằng LLM.
- **Mọi vòng lặp đều có giới hạn** và cơ chế escalate.

2. Phân tầng theo mức độ

Tầng do agent Requirement Analysis đề xuất, người dùng có thể ghi đè.

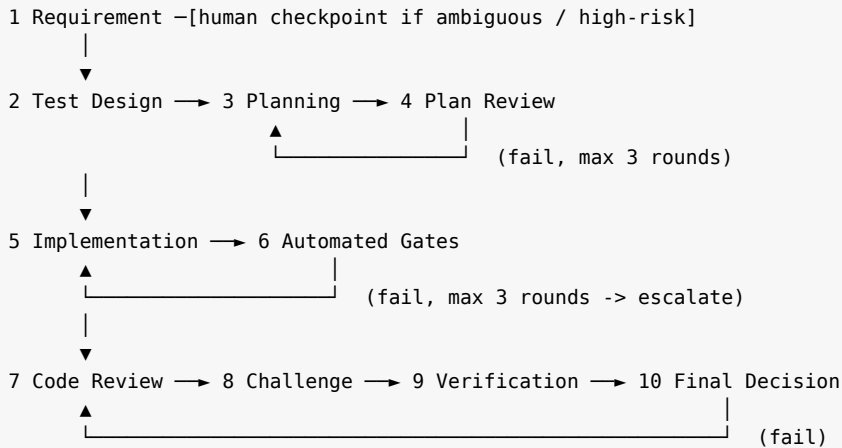
Tầng	Ví dụ	Các bước áp dụng
Trivial	Sửa typo, đổi text, config nhỏ	Implementation → Automated Gates → Diff check
Standard	Feature, bugfix thông thường	Quy trình đầy đủ (bước 1-10)
High-risk	DB migration, auth, thanh toán, xóa dữ liệu	Đầy đủ + human sign-off + security review + model thứ hai ở bước Challenge

3. Các bước thực hiện

1 Requirement Analysis	Opus
• Phân tích yêu cầu, làm rõ yêu cầu chức năng và phi chức năng, xác định edge cases. • Viết acceptance criteria dạng kiểm chứng được (Given/When/Then, có input/output cụ thể), đánh số AC-1, AC-2... • Đề xuất tầng độ khó và rủi ro. • Human checkpoint nếu yêu cầu mơ hồ hoặc task high-risk. Output: spec.md	
2 Test Design	Agent riêng, không phải người implement
• Viết test hoặc kịch bản kiểm chứng từ acceptance criteria, trước khi có code. • Lập bảng truy vết: mỗi AC ứng với ít nhất một test hoặc bằng chứng runtime. • Ghi baseline: chạy test hiện có để biết test nào đã fail sẵn. Output: test files , traceability.md , baseline.md	
3 Planning	Opus
• Thiết kế giải pháp, liệt kê file/module thay đổi, thứ tự triển khai. • Nêu rủi ro, rollback plan, thay đổi dependency và migration. Output: plan.md	

4 Plan Review	Opus, context sạch
- Reviewer chỉ nhận spec.md và plan.md, không nhận lập luận của người viết plan. - Kiểm tra: yêu cầu bị bỏ sót, assumption sai, tính khả thi, tính nhất quán, mọi AC đã có hướng kiểm chứng. - Không đạt thì quay lại bước 3 (tối đa 3 vòng, sau đó escalate lên người). Output: plan-review.md	
5 Implementation	Sonnet
- Làm đúng theo plan đã duyệt, giữ convention và architecture hiện có, không tự ý mở rộng scope. Không được sửa, xoá, skip test hoặc cấu hình CI. Nếu thấy test sai, phải báo lại. - Commit checkpoint thường xuyên. Output: code trên branch/worktree riêng	
6 Automated Gates	Script, không dùng LLM
- Build, lint, type-check, toàn bộ test, so sánh với baseline. - Kiểm tra diff của test: có test bị xoá/skip/nới assertion không. - Kiểm tra diff chỉ chứa file trong plan (hoặc có giải trình). - Fail thì quay lại bước 5. Sau 3 vòng: escalate (Sonnet → Opus, hoặc dùng và hỏi người). Output: gate-report (raw output)	
7 Code Review	Opus, context sạch
- Chỉ nhận spec, plan và diff. - Đánh giá logic, architecture, maintainability, regression, security, edge cases. - Mỗi finding gồm: mức độ (blocker / major / minor), bằng chứng (file:line hoặc repro), đề xuất sửa. Output: code-review.md	
8 Challenge	Reviewer độc lập (ưu tiên model khác với high-risk)
- Chủ động tìm cách phá implementation, không giả định đúng. - Mỗi phát hiện phải kèm bằng chứng tái hiện được (test fail, input gây lỗi, lệnh reproduce). Ý kiến không kèm repro chỉ ghi là "nghĩ vấn", không chặn task. - Khác bước 7: bước 7 đánh giá chất lượng, bước 8 tấn công. Output: challenge.md	
9 Independent Verification	Agent độc lập, chỉ đọc
- Tự chạy lại build, unit/integration test, lint, type-check. - Kiểm tra Git diff, runtime behavior, database/API/output thực tế khi liên quan. - Lưu raw output (exit code, log) làm artifact, không chỉ tóm tắt. - Điền bảng truy vết: mỗi AC → bằng chứng thực tế. Thiếu dòng nào là chưa Done. Output: evidence.md + raw logs	
10 Final Decision	Checklist xác định
- Verification thất bại hoặc còn finding blocker/major: quay lại bước 5 (tính vào giới hạn vòng lặp). - Verification thành công: task hoàn thành. - High-risk: cần người duyệt trước khi merge. Output: quyết định cuối	

4. Sơ đồ luồng



5. Definition of Done

Task chỉ hoàn thành khi có đủ bằng chứng:

- Mọi AC có ít nhất một bằng chứng tương ứng trong bảng truy vết, và bằng chứng đó pass.
- Không còn finding mức blocker/major chưa xử lý (ở bước 7 và 8).
- Build, lint, type-check thành công (có raw output).
- Toàn bộ test hiện có và test mới đều pass, không regression so với baseline, không có test bị xoá/skip/nới lỏng.
- Runtime behavior đúng yêu cầu (có bằng chứng thực tế).
- Git diff chỉ chứa file trong plan hoặc có giải trình.
- Với high-risk: có người duyệt và rollback plan đã được kiểm tra.

6. Quy tắc chung

- **Artifact có cấu trúc:** truyền tài liệu (spec.md, plan.md, evidence.md...) giữa các bước, không truyền tóm tắt bằng lời.
- **Reviewer/verifier context sạch:** không nhận lập luận hay báo cáo tự đánh giá của người thực hiện.
- **Phân vai khác nhau** cho các reviewer (correctness, security, regression, requirement coverage) để tránh chung điểm mù.
- **Giới hạn quyền agent:** không truy cập secrets, không push thẳng, không sửa CI.
- **Ghi log** lý do các lần reject để cải tiến prompt và quy trình theo thời gian.

7. Quy trình chạy song song (Parallel Agents)

Áp dụng khi task đủ lớn để chia thành nhiều phần độc lập. Mục tiêu: tăng tốc mà **không làm mất tính kiểm chứng**. Mọi nguyên tắc ở phần 1 vẫn giữ nguyên; phần này bổ sung cách chia việc, tránh xung đột và tích hợp kết quả.

7.1 Khi nào nên và không nên song song

Nên song song	Không nên song song
- Các phần có ranh giới file/module rõ ràng (ví dụ backend, frontend, docs, migration script). - Có thể định nghĩa interface/contract trước khi code. - Các phần không phụ thuộc kết quả của nhau, hoặc phụ thuộc qua contract đã đóng băng. - Các bước review và verification độc lập nhau.	- Nhiều phần cùng sửa một file hoặc cùng một hàm. - Chưa rõ kiến trúc, contract còn thay đổi. - Task nhỏ (Trivial) hoặc chi phí điều phối lớn hơn lợi ích. - Thay đổi có thứ tự bắt buộc (ví dụ migration phải chạy trước code dùng schema mới).

Các bước có thể song song ngay cả trong quy trình thường:

- Bước 2 (Test Design) và bước 3 (Planning), vì cả hai chỉ cần spec.md. Plan Review sẽ đối chiếu hai kết quả.
- Bước 7 (Code Review) và bước 8 (Challenge), nếu Challenge không đọc code-review.md để giữ tính độc lập.
- Bước 9: các suite kiểm tra (unit, integration, lint, runtime) chạy song song, một agent tổng hợp bằng truy vết.

7.2 Các bước

P1 Decomposition	Opus
- Chia task thành các work package (WP). Mỗi WP có: mục tiêu, AC liên quan, danh sách file sở hữu độc quyền, test riêng, phụ thuộc. - Vẽ đồ thị phụ thuộc (DAG) giữa các WP; xác định WP nào chạy song song, WP nào phải chờ. - Định nghĩa contract giữa các WP (function signature, API schema, kiểu dữ liệu, event). - File dùng chung (lockfile, schema, migration, config, router) giao cho đúng một WP hoặc cho Integrator. - Nếu không thể chia mà không chồng file: không song song, quay về quy trình tuần tự. Output: work-packages.md, contracts.md	
P2 Contract Freeze & Review	Opus, context sạch
- Reviewer kiểm tra: ownership có chồng nhau không, contract có thiếu không, có vòng phụ thuộc không, mọi AC đã có WP phụ trách chưa. - Sau khi duyệt, contract bị đóng băng. Muốn đổi contract phải quay lại P1 và tạm dừng các WP phụ thuộc. - Human checkpoint với task high-risk. Output: work-packages.md đã duyệt	
P3 Parallel Implementation	Sonnet × N (mỗi WP một agent)
- Mỗi agent làm trên worktree riêng, tách từ cùng một base commit. - Chỉ được sửa file trong ownership của WP. Không sửa test, CI, hay contract. - Mỗi WP tự chạy Automated Gates (bước 6) trên branch của mình. Tối đa 3 vòng, sau đó escalate. - Giới hạn số agent chạy đồng thời (khuyến nghị 3 đến 5); WP dư đợi trong hàng đợi. - Agent chỉ báo cáo trạng thái, không tự merge. Output: một branch/worktree cho mỗi WP	

P4 Integration

Integrator agent + script

- Merge các branch vào nhánh integration theo thứ tự DAG.
- Khi có conflict: không tự "chọn bên". Ghi lại; nếu nguyên nhân là chồng ownership thì quay về P1.
- Chạy **toàn bộ** gates và test tích hợp giữa các WP trên kết quả đã merge. Branch xanh chưa chắc merge xanh.
- Nếu gates fail sau merge: xác định WP gây lỗi và trả lại đúng WP đó (tính vào giới hạn vòng lặp).

Output: [integration-report.md](#)

P5 Parallel Review & Challenge

Nhiều reviewer song song, context sạch

- Review trên diff đã tích hợp, chia vai: correctness, security, regression, requirement coverage.
- Challenge chạy riêng, phải kèm bằng chứng tái hiện (như bước 8).
- Một aggregator gộp finding, loại trùng lặp, gán mỗi finding cho WP chịu trách nhiệm.
- Mỗi finding có mức độ (blocker/major/minor), bằng chứng và đề xuất sửa.

Output: [review-*.md](#), [challenge.md](#), [review-summary.md](#)

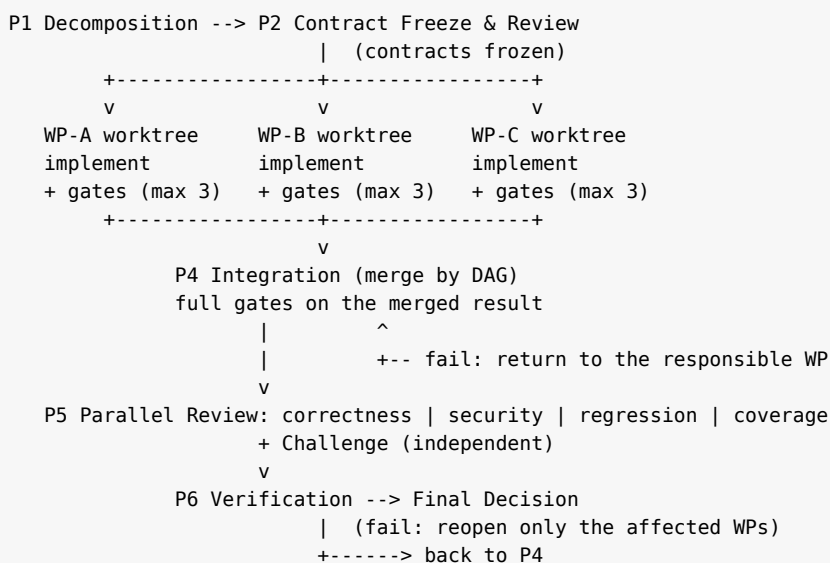
P6 Verification & Final Decision

Agent độc lập, chỉ đọc

- Thực hiện bước 9 và 10 trên kết quả đã tích hợp, không kiểm tra riêng từng branch.
- Điền bảng truy vết: mỗi AC của toàn task → bằng chứng thực tế.
- Fail: chỉ mở lại các WP liên quan, sau đó đi lại P4 → P5 → P6.

Output: [evidence.md](#) + [raw logs](#)

7.3 Sơ đồ luồng song song



7.4 Quy tắc chống xung đột

- **File ownership độc quyền:** mỗi file chỉ thuộc một WP. Script kiểm tra diff của từng branch, sửa file ngoài ownership là fail ngay.
- **Contract-first:** các WP giao tiếp qua contract đã đóng băng, không dựa vào code của WP khác đang được viết.
- **Cùng base commit:** mọi WP tách từ một commit, không rebase giữa chừng trừ khi Integrator yêu cầu.
- **Một Integrator duy nhất:** chỉ Integrator được merge và chỉ Integrator sửa file dùng chung.

- **Không chia sẻ trạng thái ngầm:** agent không đọc branch của agent khác; mọi trao đổi qua artifact.
- **Tài nguyên dùng chung:** database, cổng mạng, thư mục tạm phải tách riêng cho từng WP khi chạy test.

7.5 Xử lý sự cố

Tình huống	Hành động
Một WP fail sau 3 vòng	Escalate (Sonnet → Opus hoặc hỏi người). WP độc lập vẫn tiếp tục, WP phụ thuộc tạm dừng.
Contract sai hoặc thiếu	Dừng mọi WP phụ thuộc, quay lại P1, cập nhật contract rồi duyệt lại ở P2.
Conflict khi merge do chồng file	Quay lại P1 để tách lại hoặc gộp WP; không giải quyết conflict bằng cách đoán.
Gates fail sau merge dù từng branch đều xanh	Integrator xác định WP gây lỗi, trả lại WP đó kèm log; tính vào giới hạn vòng lặp của WP.
Vượt giới hạn agent hoặc ngân sách	Xếp hàng chờ, không mở thêm agent; ưu tiên WP nằm trên đường găng của DAG.

7.6 Definition of Done bổ sung cho chế độ song song

- Mỗi WP chỉ sửa file trong ownership của mình (có kết quả kiểm tra diff).
- Contract không bị thay đổi sau khi đóng băng, hoặc mọi thay đổi đều đã đi lại P2.
- Toàn bộ gates và test tích hợp pass trên **kết quả đã merge**, không chỉ trên từng branch.
- Bảng truy vết phủ đủ AC của toàn task, không chỉ AC của từng WP.
- Không còn conflict, WP dở dang hay branch chưa được xử lý.

8. Core Principle

**Do not trust the agent's completion report.
Trust independently verifiable evidence.**